

QRISC-V996 操作指南

开箱即跑的 RV32 Linux SoC 仿真平台

在 WSL / Linux 上,把一套从源码自编的 RV32IMA Linux 5.4 跑在 **biRISC-V** 双发射核 + 全 RTL 外设 (UART / Timer / GPIO / SPI / 中断控制器)的周期级仿真上,配套 SBI 引导器、裸机/Linux 双线 SDK、虚拟磁盘,以及一个串口控制台 GUI。

项	内容
内核	Linux 5.4.0(rv32ima, glibc)
核	biRISC-V dual-issue RV32IMA + Zicsr, sv32 MMU
外设	riscv_soc:uart_lite / timer / gpio / spi / irq_ctrl(全真 RTL)
仿真	Verilator(纯 RTL, <code>--binary --timing</code>)
引导	SBI 引导器(M 态模拟原子指令,提供 hvc0 控制台)

命名:QRISC-V996 指整个平台/发行;**biRISC-V** 指其采用的 CPU 核 (源自 ultraembedded/biriscv,保留原名以示署名)。

1. 这是什么

QRISC-V996 是一个完整的 RISC-V SoC 平台:把完全从源码自编的 RV32IMA Linux,运行在开源 biRISC-V 核 + 一整套真 RTL 外设的周期级仿真上,并通过串口控制台 GUI(或终端)与运行中的 Linux 交互。系统镜像已编好放在 `image/`,所以**不需要自己编译工具链或内核**——装好依赖、构建仿真器就能启动。想从零重建整个系统见第 9 章。

数据流

GUI / 终端 —串口字节—> `tb_soc` (Verilator: biRISC-V核 + 全RTL外设)
└─ 加载 `image/*.elf`(SBI + 内核 + `initramfs`)

内存映射(物理地址)

区域	基址	说明
DRAM	0x80000000	内核 32MB;仿真 36MB,顶部 4MB(0x82000000)给虚拟磁盘
irq_ctrl	0x90000000	Xilinx INTC
timer	0x91000000	
uart_lite	0x92000000	RX@0 TX@4 STATUS@8
spi_lite	0x93000000	CR@0x60 SR@0x64 DTR@0x68 DRR@0x6c SSR@0x70

区域	基址	说明
gpio	0x94000000	DIR@0 IN@4 OUT@8

复位向量 = 0x80000000。

2. 目录结构

QRISC-V996/	
├─ README.md / REPRODUCE.md	总览 / 完整复现指南
├─ src/	RTL 设计
│ └─ core/ top/ icache/ dcache/	biRISC-V 双发射核
│ └─ soc/	riscv_soc 外设+互联 + biriscv_soc 集成顶层
│ └─ debug/	JTAG 调试模块(jtag_dtm/dm_sba/dm_axi_master/
riscv_debug)	
├─ tb/tb_soc/	测试平台(纯 Verilog,含 JTAG remote_bitbang 桥)
├─ tools/openocd/	OpenOCD 配置 + GDB 调试说明
├─ bootloader/	SBI 引导器源码
├─ build-os/	从源码重建 OS:Makefile + 脚本 + 配置 + 内核补丁 +
dts	
├─ sdk/	写程序:baremetal/(裸机) + linux/(用户态)
├─ scripts/make_hex.py	ELF → \$readmemh hex
├─ gui/	串口控制台 GUI
├─ image/	预编译 OS 镜像(hvc0 / ttyul0)
└─ docs/	本指南

每个文件夹都有自己的 README.md,讲清该层是什么、怎么用。

3. 环境准备

操作系统:Windows 11 + WSL2(Ubuntu 建议),或任意原生 Linux。Windows 11 的 WSLg 自带图形支持,Tkinter GUI 可直接弹窗。

```
sudo apt update
sudo apt install -y verilator gtkwave device-tree-compiler \
python3 python3-tk gcc-riscv64-unknown-elf \
build-essential git
```

软件包	用途
verilator	把 Verilog RTL 编成纯 RTL 仿真二进制
gtkwave	看波形(FST)
device-tree-compiler	编设备树(dts)
python3-tk	GUI
gcc-riscv64-unknown-elf	裸机 SDK 工具链

软件包	用途
openocd(可选,0.12+)	JTAG 调试客户端(第 8.5 章)
gdb-multiarch(可选)	GDB 调试,连 OpenOCD 的 gdb server

跑预编译镜像**不需要** Linux 交叉工具链;只有从源码重建 Linux(第 9 章)才需自建 rv32ima-linux glibc 工具链。早期版本依赖的 libsystemc-dev / binutils-dev / libelf-dev **已不再需要**(测试平台改为纯 RTL)。

4. 快速开始(用编好的镜像)

```
# 4.1 构建仿真器(一次,纯 RTL,约 1-2 分钟)
cd tb/tb_soc && ./build.sh && cd ../..

# 4.2 启动 Linux — 方式 A:GUI(推荐)
python3 gui/biriscv_soc_console.py
# 模式选 Linux、选 hvc0、点 ▶启动,等到 ~ # 提示符

# 4.2 方式 B:纯命令行
./tb/tb_soc/run.sh
```

到 ~ # 后试:

```
uname -a
cat /proc/cpuinfo      # isa: rv32ima mmu: sv32 ← biRISC-V 核
ls /
```

周期级 RTL 仿真较慢,到 shell 几十秒到几分钟。**命令首字符偶尔被吃**(`ls` → `s`): 敲命令前**留一个空格**即可绕过。

5. 两个控制台变体

镜像	控制台	特点
image/...-hvc0.elf (默认)	SBI(hvc0)	轮询式, 快 ;UART 仍是真 RTL,但内核经 SBI 用它
image/...-ttyul0.elf	内核 uartlite 驱动(ttyUL0)	真中断驱动路径 (irq_ctrl + uartlite),慢但最真实

命令行切换: `ELF=image/biriscv-linux-5.4-ttyul0.elf ./tb/tb_soc/run.sh`; GUI 里「模式=Linux」下拉选。ttyUL0 经真中断(每字符一次 trap + ISR)较慢,日常用 hvc0,验证真驱动栈用 ttyUL0。

6. 写程序:裸机(无 OS,最快迭代)

直接在核上跑、直写 MMIO 操作外设,几千周期就跑起来。详见 `sdk/baremetal/README.md`。

```
cd sdk/baremetal
./build_run.sh examples/hello_uart.c      # 编译 + 跑,几秒出 UART 输出
./build_run.sh examples/gpio.c            # 翻转 GPIO
TRACE=1 ./build_run.sh examples/gpio.c    # 录波形看 gpio_out 引脚
```

写自己的: `examples/` 放个 `.c` (`#include "../bm.h"`, 写 `int main()`), `./build_run.sh examples/你的.c`。GUI 里「模式=裸机」也能选着跑。工具链用 `riscv64-unknown-elf-gcc -march=rv32ima_zicsr` (`zicsr` 才能用 `csr` 指令)。

7. 写程序:Linux 用户态(两条路)

作为进程跑,经 `/dev/mem` `mmap` 操作外设。详见 `sdk/linux/README.md`。

路 A:虚拟磁盘(推荐,改程序不重建内核)

```
cd sdk/linux
./mkdisk.sh                # 编 examples/ → tb/tb_soc/disk.hex (~几秒)
```

GUI(Linux 模式)勾「虚拟磁盘」启动 → `~ #` 后(前面留空格):

```
ls /opt                # gpio_mmap  hello
hello
gpio_mmap
```

改完程序只重跑 `./mkdisk.sh` + 重启仿真,内核不动。

原理:程序 `cpio` 放 `0x82000000`(内核 RAM 之外,tb 36MB 内存背书)。开机 `/init` 用 `vdiskcat` (经 `/dev/mem` `mmap` 读,因为 `read()` 读不到 RAM 之外)解到 `/opt`。

路 B:烤进 `initramfs`(随镜像分发,增量重建内核 ~1-2 分钟)

```
cd sdk/linux
./build_install.sh        # 编 + 装进 rootfs/usr/bin + 重建内核镜像
```

程序进 `/usr/bin`。需先自建好 Linux 工具链(第 9 章)。

8. 录波形

GUI 勾「录波形」+ 选深度 + 周期 → 启动 → 跑到周期后自停 → 「☒查看波形」开 gtkwave。

- 输出 FST 格式(比 VCD 小约 20 倍)。
- 深度 1 = 只顶层(DRAM 的 AXI / UART 串行线 / 中断,最小最快);更深看 SoC 内部接口。
- 裸机模式录波形最适合看 GPIO / SPI 引脚时序。

命令行: `TRACE=1 ./tb/tb_soc/run.sh`,或裸机 `TRACE=1 ./build_run.sh ...`。

8.5 JTAG 调试(OpenOCD / GDB)

平台带一个 **OpenOCD 兼容的 JTAG 调试模块**(RISC-V Debug Spec 0.13):JTAG TAP/DTM → DMI → Debug Module(`src/soc/debug/dm_sba.v`)。能力:**halt/resume、单步、软件断点 (ebreak,编码 0x00100073)、读写 GPR • CSR、System Bus Access(读写内存/外设,不暂停核)**。所有调试逻辑都被门控,不激活时恒 0 —— **Linux 正常启动不受影响**。完整说明与踩坑见 [tools/openocd/README.md](#)。

JTAG TAP 经 DMI 访问 Debug Module,**不占物理地址空间**;SBA 有独立 AXI 主口接 SoC 空闲 inport,可读写第 1/2 章里的整片内存映射(DRAM/uart/gpio/...)。

8.5.1 快速自测(不需要 OpenOCD)

直驱 DMI,几千周期跑完 halt→读寄存器→单步→软件断点:

```
cd tb/tb_soc
./build_vl/tb_soc +IMAGE=image.hex +DBGTEST +MAX_CYCLES=2000000
cat dbg_result.txt          # 期望 ERR=0,7 项全 PASS
./build_vl/tb_soc +IMAGE=image.hex +JTAGTEST +MAX_CYCLES=3000000  # 经真 TAP 自测,全 PASS
```

8.5.2 真 OpenOCD 客户端

```
# 终端 A:起仿真,开 JTAG 端口
cd tb/tb_soc && JTAG=9999 ./run.sh
# 终端 B:连 OpenOCD(examine 通过、gdb server 起在 :3333)
openocd -f tools/openocd/q risc-v996.cfg
```

标准命令(telnet localhost 4444): `halt / reg pc / step (PC +4) / mdw 0x80000000 / resume`。

8.5.3 GDB

```
gdb-multiarch vmlinux
(gdb) set arch riscv:rv32
(gdb) set remotetimeout 300
```

```
(gdb) target extended-remote :3333
(gdb) info reg pc sp ra a0      # 读寄存器
(gdb) x/2xw 0x80000000          # 读内存(经 SBA)
(gdb) monitor step              # 单步(仿真里比 GDB 自带 stepi 快)
```

已用 OpenOCD 0.12 + gdb-multiarch 15.1 实测端到端通过(连接/读写寄存器/读内存/单步)。仿真里 bit-bang JTAG 慢,GDB 自带 `stepi` 每步数秒,用 `monitor step` 更快;真 FPGA 上不慢。

9. 从源码重建整个 OS

外部源码(Linux 5.4 / BusyBox / 工具链)体积巨大,不入库,由 `build-os/Makefile` 拉取。详见 `build-os/README.md`。

```
# 1) 工具链:拉源码 + 自建 rv32ima-linux GCC(慢,~1 小时)
make -C build-os clone_toolchain
make -C build-os build_gcc_linux
# 2) 内核 5.4 + BusyBox 1.37.0 源码
make -C build-os clone_kernel      # torvalds/linux @ v5.4
make -C build-os clone_busybox     # busybox @ 1_37_0
# 3) 构建(用 build-os/ 的配置/补丁/dts)
make -C build-os busybox kernel image # rootfs → 内核 → 镜像 → image/
```

△ 内核必须 ≥ 5.4 :现代 rv32 glibc 用 64 位 `time_t`,需要 5.1+ 的 `time64` 系统调用;5.0 内核能启动但用户态僵死。

10. `_kernel54.sh` 自动套用的内核改动

从一份干净的 Linux 5.4 源码,构建脚本会自动套上本平台所需改动(无需手改内核源码):

1. **Xilinx INTC 驱动**:用 `build-os/kernel-patches/irq-xilinx-intc.c` 覆盖,加根中断处理器 `xil_intc_handle_irq + set_handle_irq` (5.4 RISC-V 无 `riscv,cpu-intc` 域,需自己接根)。
2. **Kconfig**:让 `XILINX_INTC` 在 RISC-V 上可选(原 5.4 仅 MicroBlaze/Zynq)。
3. **arch/riscv/Makefile** 加 `_ziclr_zifencei`; **CSR 旧名** `sbaddr→stval / sptbr→satp / mbadaddr→mtval` (适配 GCC16 / 新 binutils)。
4. **.config**:开 `XILINX_INTC / SERIAL_UARTLITE / DEVMEM / 虚拟磁盘` 相关等。
5. **rootfs**:写 `/init` (挂 `proc/sys`、解虚拟磁盘到 `/opt`、起 shell)、设备节点、`/etc/passwd`、烤入 `vdiskcat`。

此外对核本身有两处 RTL 修复(在 `src/`):AXI ID 路由(`biriscv_soc.v` 传 `ICACHE_AXI_ID=8 / DCACHE_AXI_ID=4`)、sticky-SEIP 改电平跟随(`biriscv_csr_regfile.v`)。

11. 常见问题

现象	原因 / 解决
命令首字符被吃 (<code>ls</code> → <code>s</code>)	串行注入时序边界;命令前留个空格。
<code>/opt</code> 是空的	没勾「虚拟磁盘」/ 没加 <code>+DISK</code> / 没跑 <code>mkdisk.sh</code> 。开机有「虚拟磁盘已挂载到 <code>/opt</code> 」才算挂上。
中文 / <code>ls</code> 颜色乱码	GUI 已做增量 UTF-8 解码 + 去 <code>\r</code> + 剥 ANSI;用最新 <code>gui/</code> 。
GUI 标题栏方块	WSLg 窗口管理器字体无中文字形;标题已改纯 ASCII。
GUI 窗口看不见	<code>xdotool search --name QRISC-V windowmove 60 60 windowactivate</code> 。
<code>ttyUL0</code> 很慢	真中断驱动每字符一次 <code>trap+ISR</code> ,本就慢;要快用 <code>hvc0</code> 。
<code>whoami: unknown uid 0</code>	缺 <code>/etc/passwd</code> ,已在 <code>rootfs</code> 里加(重建内核后生效)。

12. 致谢 / 许可

- CPU 核 RTL:ultraembedded/biriscv(Apache 2.0)
- SoC 外设 / 互联 RTL:ultraembedded/riscv_soc(Apache 2.0)
- SBI 引导器:ultraembedded/riscv-linux-boot(MIT)
- Linux 5.4 • BusyBox 1.37.0 • RISC-V GNU 工具链(均自源码编译)

许可文件:根目录 `LICENSE.biriscv-riscv_soc-Apache-2.0` (核/外设)、`bootloader/LICENSE.md` (引导器)。